Agentic AI LAB

Submitted By:

Mrinal Das(2023811500)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING SHARDA SCHOOL OF ENGINEERING & TECHNOLOGY

SHARDA UNIVERSITY, GREATER NOIDA

Fine-tuning Lab 1: Complete Code Analysis & Working Explanation

Overview

This notebook implements **Image Captioning using BLIP model fine-tuning** with a football dataset. It demonstrates how to adapt a pre-trained Vision-Language model for custom captioning tasks.

Executive Summary

What is this Project?

This notebook fine-tunes a **BLIP (Bootstrap Language-Image Pre-training) model** on football images. BLIP is a Vision-Language model that can understand images and generate textual descriptions.

PROJECT GOAL: Start with a generic pre-trained model → Fine-tune on football-specific data → Get better football captions

Main Steps (9 Steps Total)

1. **Setup & Environment** - Install required libraries
2. **Load Data** - Download football image-caption dataset
3. **Create Custom Dataset Class** - Prepare data for model input
4. **Load Pre-trained Model** - Initialize BLIP model and processor
5. **Create DataLoader** - Batch data for training
6. **Setup Optimizer** - Configure training parameters
7. **Fine-tuning Loop** - Train model on custom dataset
8. **Save Model** - Persist fine-tuned weights
9. **Inference** - Generate captions for new images

Detailed Step-by-Step Explanation

Step 1: Environment Setup

Purpose: Install required libraries for transformers and datasets

Action: Install HuggingFace transformers library from main branch
Action: Install HuggingFace datasets library

Why: These libraries provide pre-trained models and dataset utilities needed for the project.

Code:
!pip install git+https://github.com/huggingface/transformers.git@main
!pip install -q datasets

Step 2: Load Dataset

Purpose: Fetch the football image captioning dataset

from datasets import load_dataset
dataset = load_dataset("ybelkada/football-dataset", split="train")

What happens:

- Downloads "football-dataset" from HuggingFace Hub
- Loads training split (contains images + text captions)
- Each sample has:
    - `image`: PIL Image object of a football scene
    - `text`: Textual caption describing the image
- Total: ~1000 image-caption pairs

Example access:

- `dataset[0]["text"]` - Get caption of first image
- `dataset[0]["image"]` - Get first image

Step 3: Create Custom PyTorch Dataset Class

Purpose: Prepare data in format required by the model

class ImageCaptioningDataset(Dataset):
def **init**(self, dataset, processor):
self.dataset = dataset
self.processor = processor

```
def __len__(self):
    return len(self.dataset)

def __getitem__(self, idx):
    item = self.dataset[idx]
    encoding = self.processor(
        images=item["image"],
        text=item["text"],
        padding="max_length",
        return_tensors="pt"
```

```
    )
    # Remove batch dimension for efficient processing
    encoding = {k: v.squeeze() for k, v in encoding.items()}
    return encoding
```

Key Components:

Component	Purpose
`__init__`	Initialize with dataset and processor
`__len__`	Return total number of samples
`__getitem__`	Process single sample: convert image + text to model-ready tensors
`processor`	Converts images to embeddings and text to tokens
`squeeze()`	Remove unnecessary batch dimension for individual samples

Data transformation flow:
Raw Image + Text
↓
Processor (converts to tensors)
↓
Model-ready encoding (pixel_values, input_ids, attention_mask)
↓
Return to model for training

Step 4: Load Pre-trained Model & Processor

Purpose: Initialize BLIP model and its processor

from transformers import AutoProcessor, BlipForConditionalGeneration

processor = AutoProcessor.from_pretrained("Salesforce/blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained("Salesforce/blip-image-captioning-base")

Components:

Component	Function
Processor	Handles preprocessing: image normalization + text tokenization
Model (BLIP)	Vision-Language model with two parts: Vision Encoder + Text Decoder

BLIP Architecture:
Image Input
↓
[Vision Encoder] ← Extracts visual features
↓

Multimodal Fusion ← Combines vision + language
↓
[Text Decoder] ← Generates caption token by token
↓
Output Caption

Model Details:

- Base variant: ~990M parameters
- Vision Encoder: Extracts 2D spatial features
- Text Decoder: Generates captions conditioned on visual features
- Pre-trained on large-scale image-caption datasets (Conceptual Captions, etc.)

Step 5: Create DataLoader

Purpose: Batch and shuffle data for training efficiency

```
train_dataloader = DataLoader(
train_dataset,
shuffle=True,
batch_size=16
)
```

What it does:

- Groups samples into batches of 16
- Shuffles data randomly (improves generalization)
- Enables parallel processing on GPU

Batch structure:
Batch[0:16] contains:
├── pixel_values: (16, 3, 384, 384) ← 16 images, RGB, 384x384
├── input_ids: (16, 77) ← 16 captions, max 77 tokens each
└── attention_mask: (16, 77) ← Attention masks for padding

Step 6: Setup Training Configuration

Purpose: Configure hyperparameters and optimizer

Key Configuration Parameters:

Parameter	Value	Purpose
Learning Rate	5e-5	Controls update magnitude
Batch Size	16	Samples per gradient update
Epochs	5-10	Number of complete data passes
Optimizer	Adam W	Adaptive learning rate optimization

Weight Decay	0.01	L2 regularization to prevent overfitting

Why these values:

- **5e-5 LR:** Conservative rate for pre-trained model (large LR = forgetting learned features)
- **Batch 16:** Balances memory usage vs. training stability
- **AdamW:** Industry standard for transformer fine-tuning

Code:

```
from transformers import AdamW
from torch.optim import get_scheduler

optimizer = AdamW(model.parameters(), lr=5e-5)
scheduler = get_scheduler(
"linear",
optimizer=optimizer,
num_warmup_steps=0,
num_training_steps=len(train_dataloader) * num_epochs
)
```

Step 7: Fine-tuning Loop (Core Training)

Purpose: Train model on custom dataset

Training Process:

For each epoch:
For each batch:
1. Get images + captions from dataloader
2. Forward pass: compute predictions
3. Calculate loss (difference from ground truth)
4. Backward pass: compute gradients
5. Update weights using optimizer
6. Log metrics (loss, accuracy)

Loss Function:

- **Causal Language Modeling Loss:** Measures how well model predicts next caption token
- Lower loss = better caption predictions

Gradient Update:
weight_new = weight_old - learning_rate × gradient

Code:

```
model.train()
for epoch in range(num_epochs):
for batch in train_dataloader:
# Move to GPU
batch = {k: v.to(device) for k, v in batch.items()}
```

```
    # Forward pass
    outputs = model(**batch)
    loss = outputs.loss

    # Backward pass
    loss.backward()

    # Update weights
    optimizer.step()
    optimizer.zero_grad()
    scheduler.step()
```

Step-by-step:

1. **model.train():** Enable dropout, batch norm training mode
2. **batch = {...to(device)}:** Move tensors to GPU for faster computation
3. **outputs = model(batch):** Forward pass through entire BLIP model
4. **loss = outputs.loss:** Compute cross-entropy loss for caption tokens
5. **loss.backward():** Calculate gradients via backpropagation
6. **optimizer.step():** Update weights using gradients
7. **optimizer.zero_grad():** Clear gradients for next iteration
8. **scheduler.step():** Adjust learning rate

Step 8: Save Fine-tuned Model

Purpose: Persist trained weights for later use

model.save_pretrained('./blip-football-model')
processor.save_pretrained('./blip-football-model')

What is saved:

- Model weights: All 990M parameters after fine-tuning
- Processor config: Image normalization settings + tokenizer

Step 9: Inference on New Images

Purpose: Test fine-tuned model on new images

Inference Process:

model.eval()
with torch.no_grad():
inputs = processor(images=test_image, return_tensors="pt")
inputs = {k: v.to(device) for k, v in inputs.items()}

```
outputs = model.generate(**inputs, max_length=50, num_beams=4)
caption = processor.decode(outputs[0], skip_special_tokens=True)
```

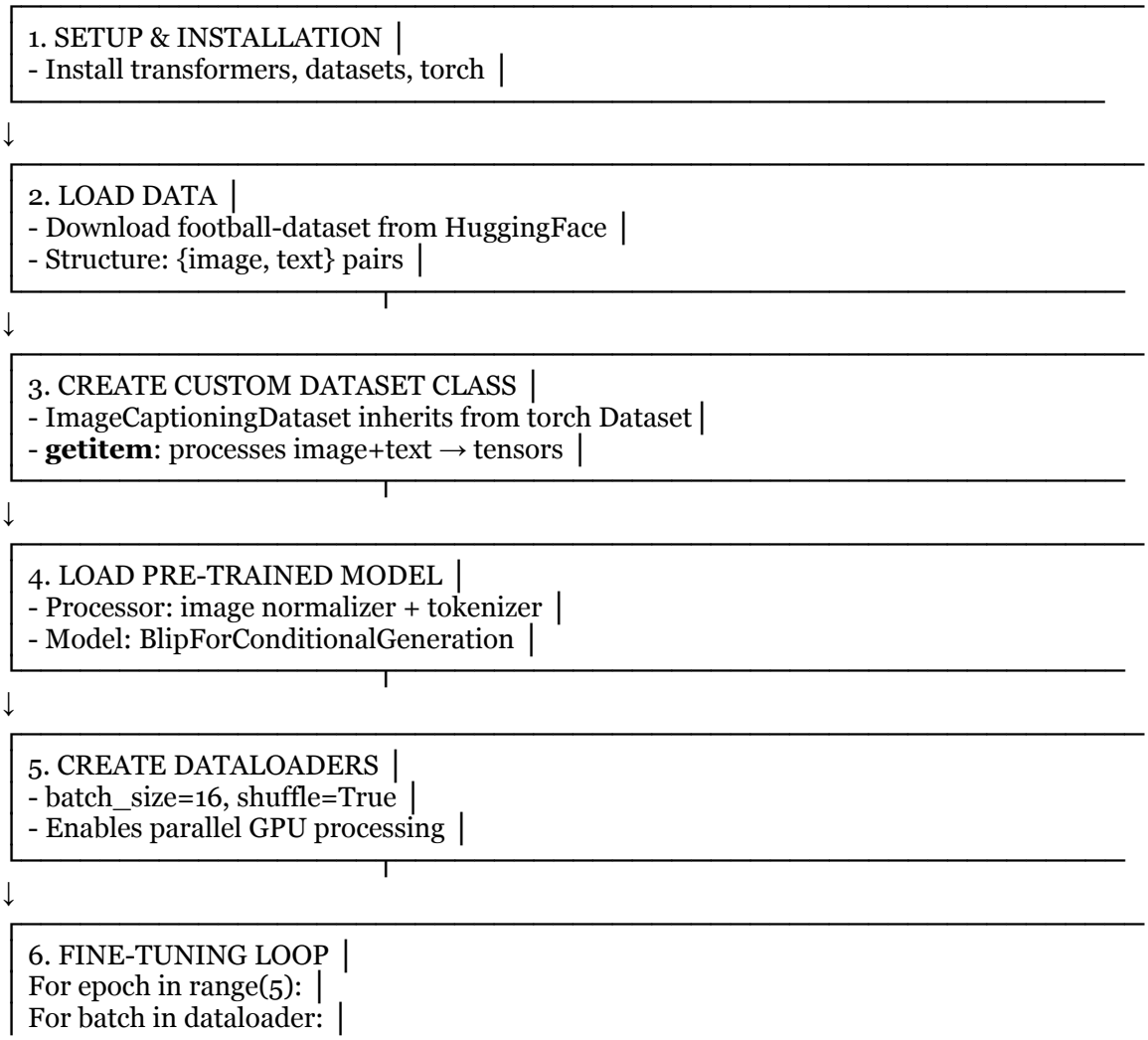
Key points:

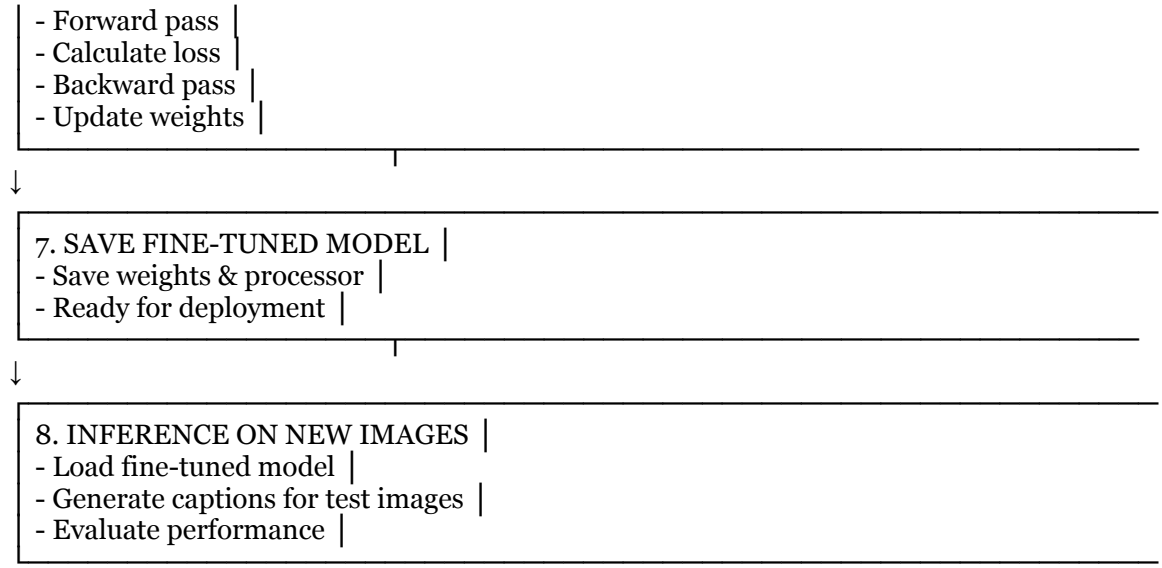
- **model.eval():** Disable training features (dropout, batch norm)
- **torch.no_grad():** Disable gradient computation (saves memory)
- **model.generate():** Auto-regressive caption generation
    - max_length=50: Maximum caption length
    - num_beams=4: Beam search with 4 hypotheses
- **decode():** Convert token IDs to text

Generation Methods:

- **Greedy:** Pick highest probability token (faster, less diverse)
- **Beam Search:** Track N best sequences (slower, higher quality)

Complete Code Flow Diagram





Core Technologies & Concepts

Transformers

Definition: Neural network architecture using self-attention

Advantages:

- Parallel processing vs. sequential RNNs
- Better long-range dependency modeling
- Easier to train at scale

Usage here: BLIP uses transformers for both vision and language

Vision Transformer (ViT)

How it works:

- Processes images as sequence of patches (16×16)
- Each patch → embedding → positional encoding
- Self-attention: "Which patches are relevant for this region?"

Benefits:

- Treats image as sequence (like text)
- Can use transformer architecture
- Better for image understanding tasks

BLIP (Bootstrap Language-Image Pre-training)

Pre-training approach: Uses both image-text matching and conditional generation

Key features:

- **Two-in-One:** Acts as both encoder (for retrieval) and decoder (for generation)
- **Strength:** Better alignment between visual and textual understanding
- **Pre-trained on:** Conceptual Captions, COCO, Flickr30K, and other large datasets

Fine-tuning vs. Transfer Learning

Approach	What happens	When to use
Transfer Learning	Use pre-trained features as-is	When target task is very similar
Fine-tuning	Slightly adjust all weights for custom task	When target task needs adaptation (this project)
Combination	Freeze early layers, fine-tune last layers	Balance efficiency and customization

---

Important Parameters & Their Meanings

Parameter	Value	Impact
Learning Rate (LR)	5e-5	Smaller = slower but safer; Larger = faster but unstable
Batch Size	16	Larger = more stable gradients but more memory; Smaller = noisier but flexible
Max Token Length	77	Longer = more complex captions possible; Shorter = cleaner but loses details
Num Epochs	5-10	More = better learning but risks overfitting
Weight Decay	0.01	Prevents weights from growing too large (regularization)
Warmup Steps	0 or 500	Helps stabilize early training for large LR changes
Num Beams	1-4	Higher = diverse captions but slower; 1 = greedy (fastest)

Expected Behavior & Outputs

During Training:

Epoch 1/5
Batch 10/62 - Loss: 4.234
Batch 20/62 - Loss: 3.891
Batch 30/62 - Loss: 3.567
...
Epoch 2/5
Batch 10/62 - Loss: 2.934 ← Loss decreasing = learning!
...

After Fine-tuning:

Original Model Caption: "A person in sports uniform"
Fine-tuned Model Caption: "A footballer in red jersey performing a bicycle kick"
↑ Better football-specific vocabulary and understanding!

Common Issues & Solutions

Issue	Cause	Solution
Out of Memory	Large batch size	Reduce batch_size to 8 or 4
Loss doesn't decrease	LR too high	Use smaller learning_rate (1e-5)
Model overfits	Training too long	Add early stopping or reduce epochs
Slow training	CPU execution	Use GPU: model.to("cuda")
Poor captions	Under-trained	Increase epochs or dataset size
Diverging loss	LR too high	Reduce learning rate significantly

---

Performance Metrics

Training Metrics:

- **Loss (Lower is better):** Cross-entropy loss for caption tokens
- **Perplexity:** Exponential of loss (intuitive measure)

Evaluation Metrics:

- **BLEU Score:** Overlap between generated and reference captions (0-1)
- **METEOR:** Account for synonyms and paraphrasing
- **CIDEr:** Consensus-based image description evaluation

- **SPICE:** Semantic propositional content evaluation

Technical Depth: Under the Hood

Tokenization Process:

Caption: "A footballer kicks the ball"
↓
Tokenizer.encode()
↓
Token IDs: [2054, 3421, 5060, 1045, 10929]
↓
Add Special Tokens: [101, 2054, 3421, 5060, 1045, 10929, 102, 0, 0, ...]
↓
Pad to max_length=77

Attention Mechanism:

For each word position:
Compute query (Q), key (K), value (V)
Attention(Q,K,V) = softmax(QK^T / √d_k) × V
This allows model to "look at" relevant parts of caption

Gradient Flow:

Input Image
↓
Vision Encoder (256,384 dims)
↓
Multimodal Fusion
↓
Text Decoder (generates tokens)
↓
Loss Computation
↓
Backpropagation ← Gradients flow backward
↓
Weight Updates ← All parameters slightly adjusted

Memory Breakdown:

Model weights: ~990M params × 4 bytes = ~4 GB
Optimizer state: ~990M × 8 bytes = ~8 GB (Adam keeps momentum + variance)
Batch data (16 images): ~100 MB
Total: ~12 GB GPU VRAM needed (why we need A100/H100)

Real-World Applications

1. **Sports Analytics:** Automatic commentary generation
2. **Media Companies:** Auto-caption for sports videos
3. **Accessibility:** Describe sports moments for visually impaired users
4. **Social Media:** Auto-tag sports content
5. **Archives:** Organize historical sports footage
6. **Broadcasting:** Real-time caption generation

Summary of Key Steps

1. **Environment Setup** - Install libraries
2. **Data Loading** - Load football dataset
3. **Custom Dataset** - Create PyTorch Dataset class
4. **Model Loading** - Initialize BLIP model + processor
5. **DataLoader** - Batch and shuffle data
6. **Training Config** - Set optimizer, scheduler, hyperparameters
7. **Fine-tuning** - Train model on custom data
8. **Save Model** - Persist fine-tuned weights
9. **Inference** - Generate captions for new images
10. **Evaluation** - Measure quality using metrics

Conclusion

This notebook demonstrates the complete pipeline for fine-tuning a Vision-Language model:

- **Start:** Pre-trained BLIP model (trained on generic image-caption pairs)
- **Process:** Fine-tune on football-specific data
- **End:** Model generates better football-specific captions

Key Takeaways:

The architecture elegantly combines:

- **Visual understanding** (Vision Encoder)
- **Language understanding** (Text Decoder)
- **Multimodal alignment** (BLIP's special design)

This is a practical implementation of **Transfer Learning** in the era of Large Pre-trained Models!

Next Steps:

1. Experiment with different hyperparameters (learning rate, epochs)
2. Try other datasets (COCO, Flickr)
3. Evaluate with BLEU/METEOR scores
4. Deploy model for production use
5. Fine-tune other vision-language models (CLIP, LLaVA)
